

MedPhysTrack

Reference Guide: Phases 1 & 2 — What Was Built and Why

This guide documents the technical setup completed in Phases 1 and 2 of the MedPhysTrack build. It is written as a reference for revisiting these steps later — especially useful if your background is in earlier-generation software development (e.g. COBOL) and you want a plain-language map of how a modern, database-driven web application is assembled.

PHASE 1 — Foundation & Database Schema

Goal

Establish the project's codebase, connect it to a cloud database, and define the full data structure (every table the application will ever use) before writing any user-facing features.

What Was Created

- **GitHub repository** — the central, version-controlled home for all project code. Every change Claude Code makes can be tracked, reviewed, and rolled back here.
- **Project scaffold** — the skeleton of a React + Vite application: configuration files (`vite.config.js`), the HTML entry point (`index.html`), and core source files (`main.jsx`, `App.jsx`, `index.css`).
- **Supabase client connection** (`src/lib/supabase.js`) — the code that allows the website to talk to its database, reading connection details from environment variables rather than being hard-coded.
- **.env.example and .gitignore** — a template for secret configuration values, and a list of files/folders that should never be uploaded to GitHub (most importantly, the real `.env` file containing actual passwords and keys).
- **Database migration files** — two SQL scripts that build the database:

File	Purpose
001_initial_schema.sql	18 tables, indexes, and automatic "updated_at" timestamp triggers, plus a trigger that auto-creates a profile record when a new user signs up.
002_rls_policies.sql	Row Level Security (RLS) rules enabled on every table, with separate access rules for the <code>super_admin</code> , <code>program_admin</code> , and <code>resident</code> roles.

Steps You Performed

- Created a free account at **supabase.com** and a new project named 'medphystrack' (choosing a database password and server region).
- Located the project's **Project URL** and **API keys** under Project Settings → API Keys (Supabase had recently introduced new 'publishable'/'secret' key naming — the older 'anon public' and 'service_role' keys, found under the 'Legacy' tab, were used instead since that's what the generated code expected).
- Created a real `.env` file (copied from `.env.example`) and filled in the Supabase URL, anon/public key, and `service_role` key.
- Confirmed `.env` was listed in `.gitignore` so secrets are never pushed to GitHub.
- Ran **001_initial_schema.sql** in the Supabase SQL Editor — first attempt returned 'relation public.profiles does not exist' because the script had an internal ordering issue. Claude Code corrected the script, and the second attempt succeeded.
- Ran **002_rls_policies.sql** successfully afterward.

- Verified in the Supabase **Table Editor** that 18 tables were created, including a few extras beyond the original plan: application_files, inquiry_logs, minutes_files, minutes_members, template_modules, and template_tasks — the last two implementing the 'copy-on-provision' curriculum template design.

***Why this matters:** Row Level Security (RLS) is the database's own gatekeeper. Even though many residency programs will eventually share the same database, RLS ensures a user from one program can never see another program's data — this is what makes the application 'multi-tenant' safely.*

Glossary — Phase 1 Terms

Repository (repo)

A project folder tracked by Git/GitHub, storing the full history of every code change.

React + Vite

React is the toolkit used to build the visual interface; Vite is the fast build tool that compiles and serves that code during development.

Supabase

A hosted backend service providing a PostgreSQL database, user authentication, file storage, and security rules — without needing to run your own server.

Migration file

A SQL script that creates or changes database structure (tables, columns, rules) in a repeatable, trackable way.

Row Level Security (RLS)

Database-enforced rules that determine which rows of a table a given user is allowed to see or modify, based on their role and organization.

Environment variables (.env)

A file holding configuration values (URLs, secret keys) kept out of the codebase and out of GitHub for security.

Copy-on-provision

The design pattern where a master "template" curriculum is copied into a new program's own tables when that program is created, so each program can customize freely without affecting the original template.

PHASE 2 — Authentication & User Roles

Goal

Allow people to securely log in, and ensure each person is routed to the correct part of the application based on their role: Super Admin, Program Admin, or Resident.

User Creation Model Decided

- **Super Admin** (you/the developer) — created manually in Supabase, no signup form needed.
- **Program Admin** — created by the Super Admin when a new residency program signs up.
- **Resident** — created/invited by their Program Admin from within the app.
- Self-serve signup was intentionally left out of this phase; all accounts begin as admin-created and the new user sets their own password via an emailed link.

What Was Created

File	Purpose
src/context/AuthContext.jsx	Manages the logged-in session and user profile across the whole app; provides a "useAuth" tool other components use to check who is logged in.

src/components/ProtectedRoute.jsx	A wrapper that blocks a page from loading unless the visitor is logged in AND has the correct role for that page.
src/pages/LoginPage.jsx	Email + password sign-in form.
src/pages/ForgotPasswordPage.jsx	Sends a password-reset email via Supabase.
src/pages/ResetPasswordPage.jsx	Lets the user set a new password after clicking the emailed link.
src/pages/dashboards/*.jsx	Three placeholder dashboards, one each for Super Admin, Program Admin, and Resident, used to confirm routing works.
src/App.jsx	Ties all the routes/pages together and decides which dashboard to show.

How Routing Works

- Visiting the site's root address (/) checks whether the visitor is logged in and what role their profile has, then automatically sends them to /super-admin, /program-admin, or /resident.
- Each of those three sections is wrapped in ProtectedRoute, which checks the required role — a Resident trying to visit /super-admin is redirected back to their own dashboard rather than seeing admin content.
- Any unrecognized address falls back to the root (/), which re-routes appropriately.

Steps You Performed

- In Supabase, went to **Authentication** → **URL Configuration** and added http://localhost:5173/reset-password to the allowed Redirect URLs list, so password-reset emails would work during local testing.
- Created the first **Super Admin account** manually: Authentication → Users → Add user, then edited that user's row in the profiles table to set role = super_admin.
- Ran the application locally with **npm run dev** (a command that starts a temporary local web server at http://localhost:5173) and confirmed:
- Logging in as each of the three roles (super_admin, program_admin, resident) showed the correct dashboard.
- Visiting another role's URL while logged in redirected back to the correct dashboard.
- The 'forgot password' flow sent an email, and clicking the link allowed setting a new password — which then automatically logged the user in (standard, expected behavior, same as Google/GitHub).
- Logging out returned to the login page, and an incorrect password showed an error rather than crashing.

***Why this matters:** This phase established the security boundary between the three types of users. Every feature built afterward (Phases 3+) sits behind this login system, so a resident can never accidentally end up on an admin screen, and vice versa.*

Glossary — Phase 2 Terms

Authentication

The process of verifying who a user is (logging in), as opposed to "authorization" (what they're allowed to do once logged in).

Session

A temporary record that the browser is "logged in" as a particular user, so they don't have to re-enter their password on every page.

Role-based routing

Sending a logged-in user to a different part of the application depending on their assigned role (super_admin / program_admin / resident).

Protected route

A page that checks login status (and role) before displaying its content, redirecting elsewhere if the check fails.

localhost:5173

The address of the temporary copy of the website running on your own computer during development — only visible to you, not the public.

Profiles table

A database table (created in Phase 1) that stores each user's role and other profile information, linked to their Supabase login account.

End of Phase 1 & 2 reference. Phase 3 (Frontend Scaffold, deployment to Vercel, and custom domain setup at medphystrack.com) is covered separately.